The Signal

VOL. 1, NO. 051

2026-03-30

FEATURES

Deliveroo's Machine Learning Platform: Powering the Future of ML

Deliveroo Engineering

Enter Deliveroo's ML Platform

For the past three years, we have been building Deliveroo's Machine Learning Platform, or the ML Platform as we like to call it. The ML Platform boosts our model-building and deployment capabilities by standardising ML workflows, streamlining the end-to-end development process and simplifying model deployment. Besides saving software engineering effort through centralising tooling, the ML Platform also reduces the time that our ML engineers spend on infrastructure tasks. As a result, our ML engineers can now iterate their ML models 2-3x faster than before.

What makes our ML Platform tick?

At the core of our ML Platform lies a carefully curated tech stack - an integrated suite of infrastructure tools, services, and libraries. It blends robust open source technologies with purpose-built, in-house components. Key open source tools include Kubernetes, Argo, and Metaflow, all seamlessly connected with leading ML frameworks like TensorFlow and PyTorch. We choose mature, community-driven solutions and actively contribute back where we can. This entire ecosystem is powered by AWS, running on EKS, and anchored by our data warehouse.

In the sections that follow, we'll take a closer look at the key components that drive our ML Platform.

To give ML engineers seamless access to scalable compute on Kubernetes, we use Metaflow , a powerful Python library that helps break down complex model-building workflows into smaller, manageable jobs. These jobs are orchestrated by Argo , one of the backbone tools in our infrastructure. One of Metaflow's biggest strengths is its flexibility. It allows engineers to move effortlessly between local development, staging on Kubernetes, and production deployment—helping teams iterate quickly as they experiment and scale. Recently, we added support for GPUs and distributed training , enabling faster training times and the ability to work with much larger datasets.

For models that require real-time predictions, we built Inferoo , our in-house inference solution. It serves models via both REST and gRPC APIs and currently handles over 1 billion requests per day . The motivation behind Inferoo was simple: to streamline model deployment and offer a consistent, reliable serving interface. Inferoo also integrates tightly with our Experimentation Platform, making it easy to run rapid A/B tests on different model versions. It also includes observability features for both batch and online models to keep an eye on model staleness, data drift and end-to-end model cost.

With Feature Store, we standardise how machine learning teams ingest and serve features for real-time inference. It supports batch ingestion from our data warehouse into Redis-backed online stores, with low-latency retrieval via a shared client library. Teams can manage feature definitions, materialisation pipelines, and versioning through a central Git-based registry. The first version of Feature Store focuses on enabling rapid experimentation and consistent deployment for live models. Future plans include real-time feature ingestion, feature sharing across teams, and support for offline training workflows.

How we work as a team

But even more important than the tech are the people who built all of it! Our team of software and ML engineers creates documentation, organises upskilling sessions, assists onboarding, maintains and monitors infrastructure, designs policies, selects external tooling and builds in-house apps such as our inference service and feature store. This year, the team is bringing all of our functionality together in an internal ML portal, which will serve as a centralised catalogue for all AI/ML models, offering visibility into model performance, system health, and operational cost metrics. Future iterations of the ML portal will also incorporate platform capabilities, to further simplify infrastructure creation and model deployment.

Three principles that we always keep in mind are automation, cohesion and self-serve:

Automation : Our goal is to develop a platform that automates repetitive tasks, keeping ML engineers focussed on building models and minimising the risk of human error.

Cohesion : We're building a unified platform that emphasises the user journey. This platform needs to seamlessly integrate the entire model lifecycle, minimising the time teams spend connecting different frameworks and onboarding models onto services.

Self-serve : We provide tooling that empower ML engineers to be self-sufficient, pushing the boundaries of technology without requiring the help of a software engineer at each turn. For example, we recently introduced support for GPUs and distributed capabilities for model training workflows. Both of these innovations significantly reduced model training time and enabled the handling of larger datasets.

What's Next?

Deliveroo's ML teams are constantly pushing the envelope, requesting new functionality for our ML Platform. We are already hosting some of the 'next generation' AI models, as well as supporting integrations between our homegrown models and externally hosted Large Language Models (LLMs). In addition, the governance of these new AI models is a big challenge that we are currently working very hard on to get right.

In future posts, we hope to share more detail on how we support these exciting new developments. So stay tuned for more updates as we continue to push the boundaries of what's possible with ML and AI.

Book Notes: The Dark Art of Linear Algebra by Seth Braver — Chapter 1 Review

Ruslan Spivak

"Mathematics is the art of reducing any problem to linear algebra." — William Stein

If you've ever looked at a vector and thought, "Just a column of numbers, right?", this chapter will change that. The Dark Art of Linear Algebra (aka DALA) by Seth Braver opens with one of the clearest intros I've read. Not every part clicks on the first pass, but the effort pays off. Paired with the author's videos, this is a strong starting point whether you're learning math for the first time or coming back to it with purpose.

As I wrote in Unlocking AI with Math and [Book Notes] Infinitesimals, Derivatives, and Beer – Full Frontal Calculus (Ch. 1) , I'm not learning math to pass a test. I'm learning it to understand the machinery behind AI and robotics, and eventually build machines of my own. (That would be fun, right?)

That goal needs a solid grasp of linear algebra. And it starts with understanding what a vector really is. Not just how to work with vectors algebraically, but how they behave in space and fit into a larger structure.

This chapter helped me sharpen that understanding.

What's a Vector?

The book makes it clear that the answer to this question will evolve as you go deeper into linear algebra. But Chapter 1 starts simple: a vector is an arrow. A geometric object. A displacement.

In the video that comes with the chapter, the author even says to forget everything you think you know about vectors. He introduces them geometrically, which makes them feel tangible and helps you see familiar algebraic ideas in a visual, spatial way.

The book introduces vector addition visually. Once you see vectors as displacements or moves through space, the addition feels natural. Almost obvious.

Image source: DALA Ch1

The text doesn't focus on vector subtraction, but there's an exercise on it. The companion video shows two methods. One of them is subtraction by addition: flip the direction of the vector you want to subtract, then add. It reminded me of that Office scene where Andy says "addition by subtraction," and Michael asks, "What does that even mean?" In that context, it's just a throwaway phrase. But in vector math, subtraction by addition is a real method. Flip the vector, then add. If you've done engineering, you've likely seen this before.

Vector addition also follows familiar rules like commutativity and associativity . If those sound fuzzy, the book and video prove them using triangles and parallelograms. No heavy algebra, just geometry.

One nice bonus is that the commutative proof gives you another way to add vectors. Place both tails at the same point, draw a parallelogram, and the diagonal gives the sum. Itss clean and easy to visualize:

Scalar multiplication is introduced as a way to stretch, shrink, or flip a vector, not just multiply its components.

The author even explains where the word scalar comes from. Numbers are called scalars because they scale vectors. I liked that he doesn't assume you already know this.

To stretch a vector, multiply by 3.

To flip it, multiply by –1.

To collapse it, multiply by 0.

It's easier to remember when you learn it by drawing instead of just computing.

The author even explains where the word scalar comes from.

Ruslan Spivak

Only after you've built a solid geometric understanding does the author introduce the standard basis vectors: i , j , and k . By then, it's clear that 2 i + 3 j + 5 k is just a weighted sum of familiar directions.

The chapter shows how to express vectors in $\mathbb{R}^2$ and $\mathbb{R}^3$ using these basis vectors, and how to rewrite them in column form.

Length of Vectors

Be sure to watch the videos that go with this chapter. They walk you through finding the length of a vector visually.

You'll start with the Pythagorean theorem to calculate the length of a vector in $\mathbb{R}^3$, then extend the idea to $\mathbb{R}^n$. The chapter also proves the general length formula when a vector is written in Cartesian coordinates. Neat.

The chapter defines the dot product using the same geometric approach as earlier sections, and it makes sense. But for me, it really clicked in the physics example where work is defined using the dot product. The author's video made it even clearer.

In the screenshot above, I underlined "Thus we see that work, viewed in a more general setting, is simply a dot product" and scribbled "watch the video" in the margin. Just a reminder that the video is a great companion to the chapter.

The text then walks through key properties: commutativity, dotting a vector with itself, the distributive property, a test for perpendicularity, and how to compute the dot product

How bad are search results? Let's compare Google, Bing, Marginalia, Kagi, Mwmbl, and ChatGPT

Dan Luu

In The birth & death of search engine optimization , Xe suggests

Here's a fun experiment to try. Take an open source project such as yt-dlp and try to find it from a very generic term like "youtube downloader". You won't be able to find it because of all of the content farms that try to rank at the top for that term. Even though yt-dlp is probably actually what you want for a tool to download video from YouTube.

More generally, most tech folks I'm connected to seem to think that Google search results are significantly worse than they were ten years ago (Mastodon poll , Twitter poll , Threads poll). However, there's a sizable group of vocal folks who claim that search results are still great. E.g., a bluesky thought leader who gets high engagement says:

i think the rending of garments about how even google search is terrible now is pretty overblown 1

I suspect what's going on here is that some people have gotten so used working around bad software that they don't even know they're doing it, reflexively doing the modern equivalent of hitting ctrl+s all the time in editors, or ctrl+a; ctrl+c when composing anything in a text box . Every adept user of the modern web has a bag of tricks they use to get decent results from queries. From having watched quite a few users interact with computers, that doesn't appear to be normal, even among people who are quite competent in various technical fields, e.g., mechanical engineering 2 . However, it could be that people who are complaining about bad search result quality are just hopping on the "everything sucks" bandwagon and making totally unsubstantiated comments about search quality.

Since it's fairly easy to try out straightforward, naive, queries, let's try some queries. We'll look at three kinds of queries with five search engines plus ChatGPT and we'll turn off our ad blocker to get the non-expert browsing experience . I once had a computer get owned from browsing to a website with a shady ad, so I hope that doesn't happen here (in that case, I was lucky that I could tell that it happened because the malware was doing so much stuff to my computer that it was impossible to not notice).

One kind of query is a selected set of representative queries a friend of mine used to set up her new computer. My friend is a highly competent engineer outside of tech and wanted help learning "how to use computers", so I watched her try to set up a computer and pointed out holes in her mental model of how to interact with websites and software 3 .

The second kind of query is queries for the kinds of things I wanted to know in high school where I couldn't find the answer because everyone I asked (teachers, etc.) gave me obviously incorrect answers and I didn't know how to find the right answer. I was able to get the right answer from various textbooks once I got to college and had access to university libraries, but the questions are simple enough that there's no particular reason a high school student shouldn't be able to understand the answers; it's just an issue of finding the answer, so we'll take a look at how easy these answers are to find. The third kind of query is a local query for information I happened to want to get as I was writing this post.

In grading the queries, there's going to be some subjectivity here because, for example, it's not objectively clear if it's better to have moderately relevant results with no scams or very relevant results mixed interspersed with scams that try to install badware or trick you into giving up your credit card info to pay for something you shouldn't pay for . For the purposes of this post, I'm considering scams to be fairly bad, so in that specific example, I'd rate the moderately relevant results above the very relevant results that have scams mixed in. As with my other posts that have some kind of subjective ranking, there's both a short summary as well as a detailed description of results, so you can rank services yourself, if you like.

In the table below, each column is a query and each row is a search engine or ChatGPT. Results are rated (from worst to best) Terrible, Very Bad, Bad, Ok, Good, and Great, with worse results being more red and better results being more blue.

The queries are:

download youtube videos

ad blocker

download firefox

Why do wider tires have better grip?

Why do they keep making cpu transistors smaller?

vancouver snow forecast winter 2023

YouTube Adblock Firefox Tire CPU Snow

Marginalia Ok Good Ok Bad Bad Bad

ChatGPT V. Bad Great Good V. Bad V. Bad Bad

Mwmbl Bad Bad Bad Bad Bad Bad

Kagi Bad V. Bad Great Terrible Bad Terrible

Google Terrible V. Bad Bad Bad Bad Terrible

Bing Terrible Terrible Great Terrible Ok Terrible

Marginalia does relatively well by sometimes providing decent but not great answers and then providing no answers or very obviously irrelevant answers to the questions it can't

Randomize HN

Dan Luu

You ever notice that there's this funny threshold for getting to the front page on sites like HN? The exact threshold varies depending on how much traffic there is, but, for articles that aren't wildly popular, there's this moment when the article is at N-1 votes. There is, perhaps, a 60% chance that the vote will come and the article will get pushed to the front page, where it will receive a slew of votes. There is, maybe, a 40% chance it will never get the vote that pushes it to the front page, causing it to languish in obscurity forever.

It's non-optimal that an article that will receive 50 votes in expectation has a 60% chance of getting 100+ votes, and a 40% chance of getting 2 votes. Ideally, each article would always get its expected number of votes and stay on the front page for the expected number of time, giving readers exposure to the article in proportion to its popularity. Instead, by random happenstance, plenty of interesting content never makes it the front page, and as a result, the content that does make it gets a higher than optimal level of exposure.

You also see the same problem, with the sign bit flipped, on low traffic sites that push things to the front page the moment they're posted, like lobste.rs and the smaller sub-reddits : they displace links that most people would be interested in by putting links that almost no one cares about on the front page just so that the few things people do care about get enough exposure to be upvoted. On reddit, users "fix" this problem by heavily downvoting most submissions, pushing them off the front page, resulting in a problem that's fundamentally the same as the problem HN has.

Instead of implementing some simple and easy to optimize, sites pile on ad hoc rules. Reddit implemented the rising page , but it fails to solve the problem. On low-traffic sub-reddits, like r/programming the threshold is so high that it's almost always empty. On high-traffic sub-reddits, anything that's upvoted enough to make it to the rising page is already wildly successful, and whether or not an article becomes successful is heavily dependent on whether or not the first couple voters happen to be people who upvote the post instead of downvoting it, i.e., the problem of getting onto the rising page is no different than the problem of getting to the top normally.

HN tries to solve the problem by manually penalizing certain domains and keywords. That doesn't solve the problem for the 95% of posts that aren't penalized. For posts that don't make it to the front page, the obvious workaround is to delete and re-submit your post if it doesn't make the front page the first time around , but that's now a ban worthy offense . Of course, people are working around that, and HN has a workaround for the workaround, and so on. It's endless. That's the problem with "simple" ad hoc solutions.

There's an easy fix, but it's counter-intuitive. By adding a small amount of random noise to the rank of an article, we can smooth out the discontinuity between making it onto the front page and languishing in obscurity. The math is simple, but the intuition is even simpler 1 . Imagine a vastly oversimplified model where, for each article, every reader upvotes with a fixed probability and the front page gets many more eyeballs than the new page. The result follows. If you like, you can work through the exercise with a more realistic model, but the result is the same 2 .

> *That doesn't solve the problem for the 95% of posts that aren't penalized.*
>
> Dan Luu

Adding noise to smooth out a discontinuity is a common trick when you can settle for an approximate result. I recently employed it to work around the classic floating point problem, where adding a tiny number to a large number results in no change, which is problem when adding many small numbers to some large numbers 3 . For a simple example of applying this, consider keeping a reduced precision counter that uses loglog(n) bits to store the value. Let countVal(x) = 2^x and inc(x) = if (rand(2^x) == 0) x++ 4 . Like understanding when to apply Taylor series, this is a simple trick that people are often impressed by if they haven't seen it before 5 .

Update : HN tried this! Dan Gackle tells me that it didn't work very well (it resulted in a lot of low quality junk briefly hitting the front page and then disappearing. I think that might be fixable by tweaking some parameters, but the solution that HN settled on, having a human (or multiple humans) put submissions that are deemed to be good or interesting into a "second chance queue" that boosts the submission onto the front page, works better than an a simple randomized algorithm with no direct human input could with any amount of parameter tweaking. I think this is also true of moderation, where the "new" dang/sctb moderation regime has resulted in a marked increase in comment quality, probably better than anything that could be done with an automated ML-based solution today — Google and FB have some of the most advanced automated systems in the world, and the quality of the result is much worse than what we see on HN.

Also, at the time this post was written (2013), the threshold to get onto the front page was often 2-3 votes, making the marginal impact of a random passerby who happens to like a submission checking the new page very large. Even during off peak times now (in 2019), the threshold seems to be much higher, reducing the amount of randomness. Additionally, the rise in the popularity of HN increased the sheer volume of low quality content that languishes on the new page, which would reduce the exposure that any particular "good" submisison would get if it were among the 30 items on the new page that would randomly get boosted onto the front page. That doesn't mean there aren't still problems with the current system: most people seem to upvote and comment based on the title of the article and not the content (to check this, read the comments of articles that are mistitled before someone calls this out for a partiular post

Docker run all the things with user namespaces

Jessie Frazelle

If you weren't aware user namepace support was added to Docker awhile back in the "Experimental" builds. But with the upcoming release of Docker Engine 1.10.0, Phil Estes is working on moving it into stable . Now this is all super exciting and blah blah blah, but what I am going to talk about today is how I started running all the containers from my Docker Containers on the Desktop with the new user namespace support. The containers/images in that post were already doing some linux-y magic, but with a little more, they are perfect. I'm not going to go through them all but I will go through some interesting ones, including even how to run Docker-in-Docker.

This one was shockingly easy. The only things I needed to add to my original command were –group-add video and –group-add audio . Makes sense right.. we obviously want to be a member of those groups to watch Taylor Swift music videos.

The full command is below. I even made a custom seccomp whitelist for chrome, you can view it in my dotfiles repo: github.com/jessfraz/dotfiles . Seccomp will be shipped in 1.10 as well, along with a default whitelist! (But I degress that is not the point of this blog post.)

$ docker run -d \ –memory 3gb \ -v /etc/localtime:/etc/localtime:ro \ -v /tmp/. X11-unix:/tmp/. X11-unix \ -e DISPLAY=unix$DISPLAY \ -v $HOME/Downloads:/root/Downloads \ -v $HOME/.chrome:/data \ -v /dev/shm:/dev/shm \ –security-opt seccomp:/etc/docker/seccomp/chrome.json \ –device /dev/snd \ –device /dev/dri \ –device /dev/video0 \ –group-add audio \ –group-add video \ –name chrome \ jess/chrome –user-data-dir=/data

Notify-osd and Irssi

Now I have always run my notifications daemon in a container, because that stuff is nasty to install, so many dependencies, ewwww. This one was a bit more tricky beacuase it involves dbus but it is a way cleaner solution than the way I was originally running it.

$ docker run -d \ -v /etc/localtime:/etc/localtime:ro \ -v /tmp/. X11-unix:/tmp/. X11-unix \ -v /etc \ -v /home/user/.dbus \ -v /home/user/.cache/dconf \ -e DISPLAY=unix$DISPLAY \ –name notify_osd \ jess/notify-osd

you can test with $ docker exec -it notify_osd notify-send hello

Not to bad right? I am creating those volumes on run so that we can then share them with our irssi container. This way when someone pings me I get a notification, duh!

$ docker run –rm -it \ -v /etc/localtime:/etc/localtime:ro \ -v $HOME/.irssi:/home/user/.irssi \ –volumes-from notify_osd \ -e DBUS_SESSION_DBUS_ADDRESS="unix:abstract=/home/user/.dbus/session-bus/$(docker exec notify_osd ls /home/user/.dbus/session-bus/)" \ –name irssi \ jess/irssi

So this is pretty simple as well even considering the real gross part is trying to get the DBUS_SESSION_DBUS_ADDRESS from our notify_osd container.

Let's get into the fun part.

Docker-in-Docker

When running the docker daemon with user namespace support, you cannot use docker run flags like –privileged , –net host , –pid host , etc. These are for pretty obvious reasons so I'm not going to get into it, if you want to know more RTFM.

Okay so we can't use –privileged , but but but that's how I run docker-in-docker... ok let's think about it. What is –privileged actually doing? Well for starters it's allowing all capabilites, but... do we really need them all? The answer is no.. all we really need is CAP_SYS_ADMIN and CAP_NET_ADMIN . So that gets us pretty far but we also need to disable the default seccomp profile (because it drops clone args, mount , and a bunch of others). Lastly we need to run with a different apparmor profile so we can have more capabilities as well.

This leaves us with:

Dockerfile $ docker run -d \ -v /etc/localtime:/etc/localtime:ro \ -v /tmp/.

Jessie Frazelle

Files are hard

Dan Luu

I haven't used a desktop email client in years. None of them could handle the volume of email I get without at least occasionally corrupting my mailbox. Pine, Eudora, and outlook have all corrupted my inbox, forcing me to restore from backup. How is it that desktop mail clients are less reliable than gmail, even though my gmail account not only handles more email than I ever had on desktop clients, but also allows simultaneous access from multiple locations across the globe? Distributed systems have an unfair advantage, in that they can be robust against total disk failure in a way that desktop clients can't, but none of the file corruption issues I've had have been from total disk failure. Why has my experience with desktop applications been so bad?

Well, what sort of failures can occur? Crash consistency (maintaining consistent state even if there's a crash) is probably the easiest property to consider, since we can assume that everything, from the filesystem to the disk, works correctly; let's consider that first.

Pillai et al. had a paper and presentation at OSDI '14 on exactly how hard it is to save data without corruption or data loss.

Let's look at a simple example of what it takes to save data in a way that's robust against a crash. Say we have a file that contains the text a foo and we want to update the file to contain a bar . The pwrite function looks like it's designed for this exact thing. It takes a file descriptor, what we want to write, a length, and an offset. So we might try

pwrite([file], "bar", 3, 2) /\/ write 3 bytes at offset 2

What happens? If nothing goes wrong, the file will contain a bar , but if there's a crash during the write, we could get a boo , a far , or any other combination. Note that you may want to consider this an example over sectors or blocks and not chars/bytes.

If we want atomicity (so we either end up with a foo or a bar but nothing in between) one standard technique is to make a copy of the data we're about to change in an undo log file, modify the "real" file, and then delete the log file. If a crash happens, we can recover from the log. We might write something like

creat(/dir/log); write(/dir/log, "2,3,foo", 7); pwrite(/dir/orig, "bar", 3, 2); unlink(/dir/log);

This should allow recovery from a crash without data corruption via the undo log, at least if we're using ext3 and we made sure to mount our drive with data=journal . But we're out of luck if, like most people, we're using the default 1 – with the default data=ordered , the write and pwrite syscalls can be reordered, causing the write to orig to happen before the write to the log, which defeats the purpose of having a log. We can fix that.

creat(/dir/log); write(/dir/log, "2, 3, foo"); fsync(/dir/log); /\/ don't allow write to be reordered past pwrite pwrite(/dir/orig, 2, "bar"); fsync(/dir/orig); unlink(/dir/log);

That should force things to occur in the correct order, at least if we're using ext3 with data=journal or data=ordered . If we're using data=writeback , a crash during the the write or fsync to log can leave log in a state where the filesize has been adjusted for the write of "bar", but the data hasn't been written, which means that the log will contain random garbage. This is because with data=writeback , metadata is journaled , but data operations aren't, which means that data operations (like writing data to a file) aren't ordered with respect to metadata operations (like adjusting the size of a file for a write).

We can fix that by adding a checksum to the log file when creating it. If the contents of log don't contain a valid checksum, then we'll know that we ran into the situation described above.

creat(/dir/log); write(/dir/log, "2, 3, [checksum], foo"); /\/ add checksum to log file fsync(/dir/log); pwrite(/dir/orig, 2, "bar"); fsync(/dir/orig); unlink(/dir/log);

That's safe, at least on current configurations of ext3. But it's legal for a filesystem to end up in a state where the log is never created unless we issue an fsync to the parent directory.

creat(/dir/log); write(/dir/log, "2, 3, [checksum], foo"); fsync(/dir/log); fsync(/dir); /\/ fsync parent directory of log file pwrite(/dir/orig, 2, "bar"); fsync(/dir/orig); unlink(/dir/log);

That should prevent corruption on any Linux filesystem, but if we want to make sure that the file actually contains "bar", we need another fsync at the end.

creat(/dir/log); write(/dir/log, "2, 3, [checksum], foo"); fsync(/dir/log); fsync(/dir); pwrite(/dir/orig, 2, "bar"); fsync(/dir/orig); unlink(/dir/log); fsync(/dir);

That results in consistent behavior and guarantees that our operation actually modifies the file after it's completed, as long as we assume that fsync actually flushes to disk. OS X and some versions of ext3 have an fsync that doesn't really flush to disk. OS X requires fcntl(F_FULLFSYNC) to flush to disk, and some versions of ext3 only flush to disk if the the inode changed (which would only happen at most once a second on writes to the same file, since the inode mtime has one second granularity), as an optimization.

Even if we assume fsync issues a flush command to the disk, some disks ignore flush directives for the same reason fsync is gimped on OS X and some versions of ext3 – to look better in benchmarks. Handling that is beyond the scope of this post, but the Rajimwale et al. DSN '11 paper and related work cover that issue.

Your dev environment matters less than you think

Itamar Turner-Trauring

How do you setup your dev environment? Depending on your language there are many choices of editor, package manager, build tool, linter, on and on. And every article you find will have a different combination of suggested tools, each of which claiming that their list is The Right Way To Do Things.

So which do you choose?

The short answer: it doesn't matter. Your choice of dev environment is meaningless.

The slightly less flippant answer is that, yes, there are some contraints on which tools you should pick, but otherwise you should just pick something and move on.

Let's see why dev environments don't matter that much in the end, and what limited constraints you should apply when choosing your tools.

Learning how to cook

Imagine you're training to become a chef. You will need to learn how to use a knife correctly, to chop and dice safely and quickly.

And yes, you need a sharp knife. But when you're starting out, it doesn't matter which knife you use: just pick something sharp and good enough, and move on. After all, the knife is just a tool.

The people eating the food you cook don't care about which knife you used: they care how the food tastes and looks.

After six months in the kitchen, you'll start understanding how you personally use a knife, what cuisines you want to pursue, what techniques you want to vary. And then you'll have the knowledge to pick a specific knife or knives exactly suited to your needs.

But remember: the people eating your food still won't care which knife you used.

Choosing a dev environment

When you use a website, you don't care which build tool the programmer used. When you run an app, you don't care which editor they used. You want the software to work, to do what it says, to be easy to use, to get out of your way—and you don't care how they did it.

And that applies just as much to the users of your code: they don't care which tools you used.

And when you're starting out, whether programming in general or a new language or framework, you don't know how you will like to work. So instead of obsessing over finding the ideal development environment and toolchain, just pick tools that are good enough:

Popular: So you can easily find help.

Easy to get going: Your goal is ship useful code, and as a beginner time spent fiddling with your dev environment won't help with that.

Once you have enough experience, you will start developing opinions. You might become choosy about which tools you use, or end up customizing them to your needs. You might even write an article about your particular dev environment and favorite tools.

But however strong your preferences are, chances are that given tools you don't quite like as much, you will still do just fine. If you know what you're doing, you can chop vegetables with any sharp knife, even if it's not your favorite.

Tired of scrambling to get your job done?

If you were productive enough, you could take the afternoon off, confident you'd produced high value work. Not to mention having an easier time finding a new job when you need one.

Learn the secret skills of productive programmers .

You want the software to work, to do what it says, to be easy to use, to get out of your way—and you don't care how they did it.

Itamar Turner-Trauring

Let's Build A Simple Interpreter. Part 19: Nested Procedure Calls

Ruslan Spivak

" What I cannot create, I do not understand." — Richard Feynman

As I promised you last time, today we're going to expand on the material covered in the previous article and talk about executing nested procedure calls. Just like last time, we will limit our focus today to procedures that can access their parameters and local variables only. We will cover accessing non-local variables in the next article.

Here is the sample program for today:

```
program Main ;

procedure Alpha ( a : integer ; b : integer ) ; var x : integer ;

procedure Beta ( a : integer ; b : integer ) ; var x : integer ;
begin x := a * 10 + b * 2 ; end ;

begin x := ( a + b ) * 2 ;

Beta ( 5 , 10 ) ; { procedure call } end ;

begin { Main }

Alpha ( 3 + 5 , 7 ) ; { procedure call }

end . { Main }
```

The nesting relationships diagram for the program looks like this:

Some things to note about the above program:

it has two procedure declarations, Alpha and Beta

Alpha is declared inside the main program (the global scope)

the Beta procedure is declared inside the Alpha procedure

both Alpha and Beta have the same names for their formal parameters: integers a and b

and both Alpha and Beta have the same local variable x

the program has nested calls: the Beta procedure is called from the Alpha procedure, which, in turn, is called from the main program

Now, let's do an experiment. Download the part19.pas file from GitHub and run the interpreter from the previous article with the part19.pas file as its input to see what happens when the interpreter executes nested procedure calls (the main program calling Alpha calling Beta):

```
$ python spi.py part19.pas –stack

ENTER: PROGRAM Main CALL STACK 1 : PROGRAM Main

...

ENTER: PROCEDURE Beta CALL STACK 2 : PROCEDURE Beta a : 5 b : 10 2 : PROCEDURE Alpha a : 8 b : 7 x : 30 1 : PROGRAM Main

LEAVE: PROCEDURE Beta CALL STACK 2 : PROCEDURE Beta a : 5 b : 10 x : 70 2 : PROCEDURE Alpha a : 8 b : 7 x : 30 1 : PROGRAM Main

...

LEAVE: PROGRAM Main CALL STACK 1 : PROGRAM Main
```

It just works! There are no errors. And if you study the contents of the ARs(activation records), you can see that the values stored in the activation records for the Alpha and Beta procedure calls are correct. So, what's the catch then? There is one small issue. If you take a look at the output where it says ' ENTER : PROCEDURE Beta', you can see that the nesting level for the Beta and Alpha procedure call is the same, it's 2 (two). The nesting level for Alpha should be 2 and the nesting level for Beta should be 3. That's the issue that we need to fix. Right now the nesting_level value in the visit_ProcedureCall method is hardcoded to be 2 (two):

```
def visit_ProcedureCall ( self , node ): proc_name = node . proc_name

ar = ActivationRecord ( name = proc_name , type = ARType . PROCEDURE , nesting_level = 2 , )

proc_symbol = node . proc_symbol ...
```

Let's get rid of the hardcoded value. How do we determine a nesting level for a procedure call? In the method above we have a procedure symbol and it is stored in a scoped symbol table that has the right scope level that we can use as the value of the nesting_level parameter (see Part 14 for more details about scopes and scope levels).

How do we get to the scoped symbol table's scope level through the procedure symbol?

Let's look at the following parts of the ScopedSymbolTable class:

```
class ScopedSymbolTable : def __init__ ( self , scope_name , scope_level , enclosing_scope = None ): ... self . scope_level = scope_level ...

def insert ( self , symbol ): self . log ( f 'Insert: { symbol . name } ' ) self . _symbols [ symbol . name ] = symbol
```

Looking at the code above, we can see that we could assign the scope level of a scoped symbol table to a symbol when we store the symbol in the scoped symbol table (scope) inside the insert method. This way we will have access to the procedure symbol's scope level in the visit_Procedure method during the interpretation phase. And that's exactly what we need.

Here and Now: Reusing Code at Feedzai with JupyterLab Snippets

João Palmeiro · Feedzai Tech

Data scientists use different Jupyter notebooks every day — ranging from disposable ones for quick tasks to those shareable with clients. Over time, more and more notebooks accumulate, making it increasingly difficult to reuse them in whole or in part. To mitigate this problem and make the most relevant pieces of code quickly accessible to every data scientist, we developed JupyterLab Snippets at Feedzai — our take on leveraging code snippets directly on JupyterLab.

JupyterLab is a computational notebook platform that enables us to carry on data science work (and beyond) via notebooks. These notebooks, where code, text, and images come together, allow us to complete all kinds of tasks, keeping each input close to each output. At Feedzai, data scientists have access to different JupyterLab environments packed with custom notebooks and Python packages. Here they go from analyzing data to training models, from preparing reports to debugging the system — a "lab of all trades" , we could say.

Given the importance of JupyterLab and notebooks in the daily work of our data scientists, we started a research project that culminated in JupyterLab Snippets. First, we collected statistics from internal notebook repositories across teams and conducted user interviews with junior and senior data scientists. We needed to know more about how data scientists actually use JupyterLab and notebooks, and what their ideas are for a better platform.

A sneak peek of JupyterLab Snippets. From the very start, every data scientist can import snippets immediately or start their collection by creating one from scratch.

After compiling the insights, several aspects became clear:

JupyterLab and notebooks are heavily used for all sorts of data science and ad-hoc tasks (typically one per notebook). There is no single type of task (e.g., exploratory data analysis) that is more commonly addressed with notebooks than others.

While file structures may look similar at first glance (each environment has at least a model training notebook, for example), data scientists have different and personal workflows for coding notebooks.

Duplicating past notebooks available in the environment or getting them from repositories is a very common practice.

Notebooks composed of code snippets is also an identified practice. These notebooks are not intended to be run, and contain code and text for future reference (imagine a notepad in notebook format). This case and the previous one imply removal of unnecessary parts and parameterization.

Data scientists regularly communicate within teams and individually. Sharing notebooks/snippets is a common practice.

No significant issues were identified with the JupyterLab/ notebook interface.

It is true that data scientists have their own ways of finding whatever they need for their next notebook, and AI coding assistants are proliferating. Even so, would it be beneficial to improve the experience of reusing code in JupyterLab? After analyzing all the insights, we believe so. We believe in an interface that allows every data scientist to easily manage and use "their snippets" now and in one year, while also facilitating their sharing with other data scientists. JupyterLab Snippets aims to improve the developer experience, focusing on productivity and satisfaction, while having a low maintenance cost. Here is JupyterLab Snippets in its essence.

Introducing JupyterLab Snippets

JupyterLab Snippets is a JupyterLab extension for data scientists to create, manage, and use code snippets directly on JupyterLab, facilitating code reuse in their notebooks. To be more precise, the interface is a JupyterLab extension and the backend, for managing the snippet data, is a Jupyter Server one . Both communicate via a REST API.

Two of the three main parts: Snippet Sidebar and Snippet Editor. The snippets are adapted from the Vega-Altair documentation examples .

The interface, a part of JupyterLab like notebooks and the like, is divided into three main components:

Snippets Sidebar : The central hub for JupyterLab Snippets, available as a new tab in the right sidebar of JupyterLab. It allows the data scientist to manage and utilize the code snippets. Each snippet is available on a Snippet Card .

Snippet Editor : The form-like interface for creating new snippets and editing existing ones. When opened, it is available as a tab in the main work area of JupyterLab.

Shortcuts : Options for creating snippets from notebook cells. These options are available from the context menu after right-clicking on a cell, and the option to start a snippet from a cell is also available from the cell toolbar .

Snippet Cards and Shortcuts. Shortcuts consist of new cell context menu options and a new option in the cell toolbar.

In the Snippets Sidebar , in addition to the Personal collection, data scientists will find a set of pre-made snippets: the Feedzai collection. They can switch between both (or view them all at once) by selecting the desired collection in the toolbar just above the list of available snippets. The Feedzai collection is a set of snippets for common actions performed in notebooks, prepared with the help of our data scientists. This collection is specially designed for newcom-

Startup options v. cash

Dan Luu

I often talk to startups that claim that their compensation package has a higher expected value than the equivalent package at a place like Facebook, Google, Twitter, or Snapchat. One thing I don't understand about this claim is, if the claim is true, why shouldn't the startup go to an investor, sell their options for what they claim their options to be worth, and then pay me in cash? The non-obvious value of options combined with their volatility is a barrier for recruiting.

Additionally, given my risk function and the risk function of VCs, this appears to be a better deal for everyone. Like most people, extra income gives me diminishing utility, but VCs have an arguably nearly linear utility in income. Moreover, even if VCs shared my risk function, because VCs hold a diversified portfolio of investments, the same options would be worth more to them than they are to me because they can diversify away downside risk much more effectively than I can. If these startups are making a true claim about the value of their options, there should be a trade here that makes all parties better off.

In a classic series of essays written a decade ago, seemingly aimed at convincing people to either found or join startups, Paul Graham stated "If you wanted to get rich, how would you do it? I think your best bet would be to start or join a startup. That's been a reliable way to get rich for hundreds of years" and "Risk and reward are always proportionate." This risk-reward assertion is used to back the claim that people can make more money, in expectation, by joining startups and taking risky equity packages than they can by taking jobs that pay cash or cash plus public equity. However, the premise — that risk and reward are always proportionate — isn't true in the general case. It's basic finance 101 that only assets whose risk cannot be diversified away carry a risk premium (on average). Since VCs can and do diversify risk away, there's no reason to believe that an individual employee who "invests" in startup options by working at a startup is getting a deal because of the risk involved. And by the way, when you look at historical returns, VC funds don't appear to outperform other investment classes even though they get to buy a kind of startup equity that has less downside risk than the options you get as a normal employee.

So how come startups can't or won't take on more investment and pay their employees in cash? Let's start by looking at some cynical reasons, followed by some less cynical reasons.

Cynical reasons

One possible answer, perhaps the simplest possible answer, is that options aren't worth what startups claim they're worth and startups prefer options because their lack of value is less obvious than it would be with cash. A simplistic argument that this might be the case is, if you look at the amount investors pay for a fraction of an early-stage or mid-stage startup and look at the extra cash the company would have been able to raise if they gave their employee option pool to investors, it usually isn't enough to pay employees competitive compensation packages. Given that VCs don't, on average, have outsized returns, this seems to imply that employee options aren't worth as much as startups often claim. Compensation is much cheaper if you can convince people to take an arbirary number of lottery tickets in a lottery of unknown value instead of cash.

Some common ways that employee options are misrepresented are:

Strike price as value

A company that gives you 1M options with a strike price of $10 might claim that those are "worth" $10M. However, if the share price stays at $10 for the lifetime of the option, the options will end up being worth $0 because an option with a $10 strike price is an option to buy the stock at $10, which is not the same as a grant of actual shares worth $10 a piece.

Public valuation as value

Let's say a company raised $300M by selling 30% of the company, giving the company an implied valuation of $1B. The most common misrepresentation I see is that the company will claim that because they're giving an option for, say, 0.1% of the company, your option is worth $1B * 0.001 = $1M. A related, common, misrepresentation is that the company raised money last year and has increased in value since then, e.g., the company has since doubled in value, so your option is worth $2M. Even if you assume the strike price was $0 and and go with the last valuation at which the company raised money, the implied value of your option isn't $1M because investors buy a different class of stock than you get as an employee.

There are a lot of differences between the preferred stock that VCs get and the common stock that employees get; let's look at a couple of concrete scenarios.

Let's say those investors that paid $300M for 30% of the company have a straight (1x) liquidation preference, and the company sells for $500M. The 1x liquidation preference means that the investors will get 1x of their investment back before lowly common stock holders get anything, so the investors will get $300M for their 30% of the company. The other 70% of equity will split $200M: your 0.1% common stock option with a $0 strike price is worth $285k (instead of the $500k you might expect it to be worth if you multiply $500M by 0.001).

The preferred stock VCs get usually has at least a 1x liquidation preference. Let's say the investors had a 2x liquidation preference in the above scenario. They would get 2x their investment back before the common stockholders split the rest of the company. Since 2 * $300M is greater than $500M, the investors would get everything and the remaining equity holders would get $0.

Transcript of Elon Musk on stage with Dave Chapelle

Dan Luu

This is a transcription of videos Elon Musk's appearance on stage with Dave Chapelle using OpenAI's Whisper model with some manual error corrections and annotations for crowd noise.

As with the Exhibit H Twitter text message release , there are a lot of articles that quote bits of this, but the articles generally missing a lot of what happened and often paint a misleading picture of happened and the entire thing is short enough that you might as well watch or read it instead of reading someone's misleading summary. In general, the media seems to want to paint a highly unflattering picture of Elon, resulting in articles and virtual tweets that are factually incorrect. For example, it's been widely incorrectly reported that, during the "I'm rich, bitch" part, horns were played to drown out the crowd's booing of Elon, but the horn sounds were played when the previous person said the same thing, which was the most cheered statement that was recorded. The sounds are much weaker when Elon says "I'm rich, bitch" and can't be heard clearly, but it sounds like a mix of booing and cheering. It was probably the most positive crowd response that Elon got from anything and it seems inaccurate in at least two ways to say that horns were played to drown out the booing Elon was receiving. On the other hand, even though the media has tried to paint as negative a picture of Elon as possible, it's done quite a poor job and a boring, accurate, accounting of what happened in many of other sections are much less flattering than the misleading summaries that are being passed around.

Video 1

Dave : Ladies and gentlemen, make some noise for the richest man in the world.

Crowd : [mixed cheering, clapping, and boos; boos drown out cheering and clapping after a couple of seconds and continue into next statements]

Dave : Cheers and boos, I say

Crowd : [brief laugh, boos continue to drown out other crowd noise]

Dave : Elon

Crowd : [booing continues]

Elon : Hey Dave

Crowd : [booing intensifies]

Elon : [unintelligible over booing]

Dave : Controversy, buddy.

Crowd : [booing continues; some cheering can be heard]

Elon : Weren't expecting this, were ya?

Dave : It sounds like some of them people you fired are in the audience.

Crowd : [laughs, some clapping can be heard]

Elon : [laughs]

Crowd : [booing resumes]

Dave : Hey, wait a minute. Those of you booing

Crowd : [booing intensifies]

Dave : Tough [unintelligible due to booing] sounds like

Dave : You know there's one thing. All those people are booing. I'm just. I'm just pointing out the obvious. They have terrible seats. [unintelligible due to crowd noise]

Crowd : [weak laughter]

Dave : All coming from wayyy up there [unintelligible] last minute non-[unintelligible] n*****. Booo. Boooooooo.

Crowd : [quiets down]

Dave : Listen.

Crowd : [booing resumes]

Dave : Whatever. Look motherfuckas. This n**** is not even trying to die on earth

Crowd : [laughter mixed with booing, laughter louder than boos]

Video 2

Dave : His whole business model is fuck earth I'm leaving anyway

Crowd : [weak laughter, weak mixed sounds]

Dave : Do all you want. Take me with you n**** I'm going to Mars

Crowd : [laughter]

Dave : Whatever kind of pussy they got up there, that's what we'll be doin

Crowd : [weak laughter]

Dave : [laughs] Anti-gravity titty bars. Follow your dreams bitch and the money just flow all over the room

Crowd : [weak laughter]

Elon : [laughs]

Crowd : [continued laughter drowned out by resumed boo-

Personal Infrastructure

Jessie Frazelle

This post is kind of like “part two” on my series on all the weird things I do for my personal infrastructure. If you missed “part one”, you should check out Home Lab is the Dopest Lab .

I run a lot of little things to make my life easier, like a CI, some bots, and a bunch of services just for the lolz. This post will go over all of those. These run scattered across my NUCs and the cloud.

Let’s start with the most useful.

I host my own continuous integration server. Yes, you guessed it... it’s Jenkins. I use the Jenkins DSL plugin to keep everything in sync. You can find all my DSLs in my repo github.com/jessfraz/jenkins-dsl . This has all the configurations for views, keeps forks up to date, mirrors all my repositories to private git (more on this in git), builds all Dockerfiles to push to Docker Hub and my private registry (more on this in private docker registry) and a bunch of maintenance scripts.

The Makefile in this repo calls out to bash scripts which generate new DSLs for any new GitHub repos I create. Yep I even generate the automation...

There’s a bunch of other fun things in there as well that you can discover by poking around yourself.

I host my own postfix server alongside Jenkins. You can find the postfix docker image at r.j3ss.co/postfix or the Dockerfile . It’s super minimal and less gross than literally every other postfix image in existence.

You can run it with:

$ docker run –restart always -d \ –name postfix \ –net container:jenkins \ -e “ROOT_ALIAS=root@blah.com” \ -e “RELAY=[smtp-relay.gmail.com]:587” \ -e “TLS=1” \ -e “MY_DESTINATION=...., localhost” \ -e “MAILNAME=blah.com” \ r.j3ss.co/postfix

I host my own private docker registry with my own notary server and authentication server. Why? Well because about 4 years ago when I started using docker, Docker Hub was super slow and I came to love having my own super fast one.

I still push all the images to both Docker Hub and my registry and both are signed so it’s really like I am using Docker Hub as my backup. Yay, highly available... just kidding.

I made a pretty shitty UI for it. You can play with it at r.j3ss.co . The UI is from my reg project but the server component lives in the server subdirectory .

The really nice thing about both the reg command line and server is that you can get a list of CVEs on an image.

I do this by hosting my own instance of CoreOS’s Clair .

Most of my Dockerfiles live at github.com/jessfraz/dockerfiles if you are curious.

I also went over all of this on my talk on Over Engineering my Laptop / Container Linux on the Desktop . This includes all the reasons why I have continuous integration as well.

I have a script to cleanup the registry of old images clean-registry . This deletes old registry blobs that are not used in the latest version of the tag. I don’t really care about old images and I don’t want to have a huge registry filled with old shit. There is a jenkins DSL to run this.

I host my own git server. You can find the gitserver docker image at r.j3ss.co/gitserver or the Dockerfile .

You can run it with:

$ docker run –restart always -d \ –name gitserver \ -p 127.0.0.1:22:22 \ -e “PUBKEY=$(cat /.ssh/authorized_keys)” \ -v “/mnt/disks/gitserver:/home/git” \ r.j3ss.co/gitserver

It has it’s own UI that is run with Gitiles. You can find the Gitiles docker image at r.j3ss.co/gitiles or the Dockerfile .

You can run it with:

$ docker run –restart always -d \ –name gitiles \ -p 127.0.0.1:8080:8080 \ -e BASE_GIT_URL=“git@git.blah.com” \ -e SITE_TITLE=“git.blah.com” \ -v “/mnt/disks/gitserver:/home/git” \ -w /home/git \ r.j3ss.co/gitiles

ghb0t

This is one of my most useful things. It’s a GitHub Bot to automatically delete your fork’s branches after a pull request has been merged.

I am so OCD about keeping git repos clean and this is my little helper.

Check out the repo: github.com/jessfraz/ghb0t .

I go to fork your thing and there is like 300 branches my face is like pic.twitter.com/JpdpO447KS

— jessie frazelle (@jessfraz) January 23, 2017

IRC Bouncer

I host my own IRC Bouncer with ZNC. You can find the ZNC docker image at r.j3ss.co/znc or the Dockerfile .

You can run it with:

$ docker run –restart always -d \ –name znc \ -p 6697:6697 \ -v “/mnt/disks/znc:/home/user/.znc” \ r.j3ss.co/znc

upmail

This service provides email notifications for sourcegraph/

Are closed social networks inevitable?

Dan Luu

This is an archive of an old Google Buzz conversation (circa 2010?) on a variety of topics, including whether or not it's inevetible that a closed platform will dominate social.

Piaw : Social networks will be dominated primarily by network effects. That means in the long run, Facebook will dominate all the others.

Rebecca : ... which is also why no one company should dominate it. "The Social Graph" and its associated apps should be like the internet, distributed and not confined to one company's servers. I wish the narrative surrounding this battle was centered around this idea, and not around the whole Silicon Valley "who is the most genius innovator" self-aggrandizing unreality field. Thank god Tim Berners Lee wasn't from Silicon Valley, or we wouldn't have the Internet the way we know it in the first place.

I suppose I shouldn't be being so snarky, revealing how much I hate your narratives sometimes. But I think for once this isn't, as it usually is, merely harmlessly cute and endearing - you all collectively screwing up something actually important, and I'm annoyed.

Piaw : The way network effects work, one company will control it. It's inevitable.

Rebecca : No it is not inevitable! What is inevitable is either that one company controls it or that no company controls it. If you guys had been writing the narrative of the invention of the internet you would have been arguing that it was inevitable that the entire internet live on one companies servers, brokered by hidden proprietary protocols. And obviously that's just nuts.

Piaw : I see, the social graph would be collectively owned. That makes sense, but I don't see why Facebook would have an incentive to make that happen.

Rebecca : Of course not! That's why I'm biting my fingernails hoping for some other company to be the white knight and ride up and save the day, liberating the social graph (or more precisely, the APIs of the apps that live on top of them) from any hope of control by a single company. Of course, there isn't a huge incentive for any other company to do it either — the other companies are likely to just gaze enviously at Facebook and wish they got there first. Tim Berners Lee may have done great stuff for the world, but he didn't get rich or return massive value to shareholders, so the narrative of the value he created isn't included in the standard corporate hype machine or incentives.

Google is the only company with the right position, somewhat appropriate incentives, and possibly the right internal culture to be the "Tim Berners Lee" of the new social internet. That's what I was hoping for, and I'm am more than a bit bummed they don't seem to be stepping up to the plate in an effective way in this case, especially since they are doing such a fabulous job in an analogous role with Android.

Rebecca : There is a worldview lurking behind these comments, which perhaps I should try to explain. I'm been nervous about this because it contains some strange ideas, but I'm wondering what you think.

Here's a very strange assertion: Mark Zuckerberg is not a capitalist, and therefore should not be judged by capitalist logic. Before you dismiss me as nuts, stop and think for a minute. What is the essential property that makes someone a capitalist?

For instance, when Nike goes to Indonesia and sets up sweatshops there, and if communists, unhappy with low pay & terrible conditions, threaten to rebel, they are told "this is capitalism, and however noxious it is, capitalism will make us rich, so shut up and hold your peace!" What are they really saying? Nike brings poor people sewing machines and distribution networks so they can make and sell things they could not make and sell otherwise, so they become more productive and therefore richer. The productive capacity is scarce, and Nike is bringing to Indonesia a piece of this scarce resource (and taking it away from other people, like American workers.) So Indonesia gets richer, even if sweatshop workers suffer for a while.

So is Mark Zuckerberg bringing to American workers a piece of scarce productive capacity, and therefore making American workers richer? It is true he is creating for people productive capacity they did not have before — the possibility of writing social apps, like social games. This is "innovation" and it does make us richer.

But it is not wealth that follows the rules of capitalist logic. In particular, this kind of wealth of productive capacity, unlike the wealth created by setting up sewing machines, does not have the kind of inherent scarcity that fundamentally drives capitalist logic. Nike can set up its sewing machines for Americans, or Indonesians, but not for everyone at once. But Tim Berners Lee is not forced to make such choices – he can design protocols that allow everyone everywhere to produce new things, and he need not restrict how they choose to do it.

But – here's the key point – though there is no natural scarcity, there may well be "artificial" scarcity. Microsoft can obfuscate Windows API's, and bind IE to Windows. Facebook can close the social graph, and force all apps to live on its servers. "Capitalists" like these can then extract rents from this artificial scarcity. They can use the emotional appeal of capitalist rhetoric to justify their rent-seeking. People are very conditioned to believe that when local companies get rich American workers in general will also get rich – it works for Indonesia so why won't it work for us? And Facebook and Microsoft employees are getting richer. QED.

But they aren't getting richer in the same way that sweatshop employees are getting richer. The sweatshop employ-

Top lessons to learn from refactoring code with a headless CMS

Gary Butler · Harry's Engineering

The Harry's European website (for Harry's customers in the UK and the rest of Europe), like most websites, is a content-heavy website providing customers with information about our products and services, which means we seek to keep our content management separate from our codebase to enable both to change independently of each other as well as allowing content to be edited without any engineering intervention.

Having worked on a variety of websites using different Content Management Systems (CMS) over the years from WordPress, to Drupal, to SiteCore among others, there is always the challenge of deciding on good abstractions to build between the code I write while not being constrained by what's possible within the CMS. The tricky part is striking the right balance between being fully dynamic and being overly complicated resulting in the maintenance of the codebase becoming a nightmare for the team.

What is a Headless CMS?

What makes a particular CMS "headless" is in regards to how you implement it into your codebase. Unlike traditional Content Management Systems, a headless CMS is agnostic to your codebase, focusing only on the data you want. Returning the data as JSON on a request. This allows more freedom and flexibility for the developer who is not limited by the access library or any templating language.

We're currently using DatoCMS as our headless content management system, which basically means DatoCMS provides a useful dashboard and portal for managing content which enables our Product, Design and Marketing partners to manage content for the website without having to write or know any code. We can then access that content via APIs that DatoCMS provides to pull into our site at build time. This is different to some traditional CMSs because of this decoupling and doesn't prescribe a particular architecture for your site (for example, WordPress or Drupal).

Trial Builder Model within DatoCMS

NOTE: It's worth mentioning at this point that our Harry's European website is currently a statically generated JavaScript-based Single-Page Application (SPA) written in TypeScript with Gatsby and React.

What is the Trial Builder?

One of the most important flows that we have on our site is the 'Trial Builder' which is the name given to the customer journeys where you subscribe for a Harry's trial set (e.g. razor handle, blades and shave gel) and then decide on your subscription cadence for refill blades as well as other products.

See it for yourself, here: https://www.harrys.com/en/gb/signup/customize?step=1

This flow references no less than 15 pieces of content in DatoCMS made of approx, 10 images and 25 strings. We also invest a lot of time in experimenting with A/B tests in this customer journey to further optimise it to surface additional features so that customers can get the most benefit from Harry's products. We can then measure the impact on our top ecommerce funnel metrics such as conversion rate (CVR), average order value (AOV) and bill-through rate (BTR).

The Trial Builder is also leveraged by the Subscription Signup flow. They both have a similar customer journey but with an alteration to the steps or content. Though the Trial Builder is the most important of these flows, any changes need to support the other journey to facilitate code reuse. This leads to requirements for the logic to be flexible to handle these variations. With multiple variations, continuous A/B tests and changes based on country, we can quickly start to gain tech debt, spaghetti code and a generally bloated codebase.

What problem are we solving?

As mentioned earlier, the engineering challenge for working with Headless CMSs is to strike the right balance between code maintainability and flexibility (or dynamism). You want to be able to realise the ideas that the Product Managers come up with without having to re-architect your website every time but you also want to make sure you don't over-engineer your code when it's likely you'll never need 100% full flexibility either.

Back in 2023, we got this balance wrong. We attempted to make the most flexible code possible to handle as many possibilities as we could think of but the reality was we never made use of it. We had a clever setup which enabled entire subscription flows to be created from DatoCMS. From how many steps, to what elements would appear on the page. The idea was to remove any engineering effort when setting up changes or new alternative versions of the journey. This did work well at first, the complexity came when we started to make UI changes based on A/B tests by locale. Implementing these changes without breaking the complex dynamic logic became incrementally more difficult over time. Eventually, it started to block requirements.

We decided to make the code interacting with the DatoCMS APIs as flexible as possible but by doing so it caused the codebase to become the opposite.

Though the intention was to enable a customer journey to be completely configurable via the CMS, over time most of the UI changes still needed engineering implementation, but now each change had to work around this complex condition logic setup in the codebase to support our super dynamic relationship with DatoCMS.

Over time this caused each update to take longer and even blocked us from even implementing some features. As the

HN: the good parts

Dan Luu

HN comments are terrible . On any topic I'm informed about , the vast majority of comments are pretty clearly wrong . Most of the time, there are zero comments from people who know anything about the topic and the top comment is reasonable sounding but totally incorrect. Additionally, many comments are gratuitously mean. You'll often hear mean comments backed up with something like "this is better than the other possibility, where everyone just pats each other on the back with comments like 'this is great'", as if being an asshole is some sort of talisman against empty platitudes. I've seen people push back against that; when pressed, people often say that it's either impossible or inefficient to teach someone without being mean, as if telling someone that they're stupid somehow helps them learn. It's as if people learned how to explain things by watching Simon Cowell and can't comprehend the concept of an explanation that isn't littered with personal insults. Paul Graham has said, " Oh, you should never read Hacker News comments about anything you write ". Most of the negative things you hear about HN comments are true.

And yet, I haven't found a public internet forum with better technical commentary. On topics I'm familiar with, while it's rare that a thread will have even a single comment that's well-informed, when those comments appear, they usually float to the top. On other forums, well-informed comments are either non-existent or get buried by reasonable sounding but totally wrong comments when they appear, and they appear even more rarely than on HN.

By volume, there are probably more interesting technical "posts" in comments than in links. Well, that depends on what you find interesting, but that's true for my interests. If I see a low-level optimization comment from nkurz, a comment on business from patio11, a comment on how companies operate by nostrademons, I almost certainly know that I'm going to read an interesting comment. There are maybe 20 to 30 people I can think of who don't blog much, but write great comments on HN and I doubt I even know of half the people who are writing great comments on HN 1 .

I compiled a very abbreviated list of comments I like because comments seem to get lost. If you write a blog post, people will refer it years later, but comments mostly disappear. I think that's sad – there's a lot of great material on HN (and yes, even more not-so-great material).

What's the deal with MS Word's file format ?

Basically, the Word file format is a binary dump of memory. I kid you not. They just took whatever was in memory and wrote it out to disk. We can try to reason why (maybe it was faster, maybe it made the code smaller), but I think the overriding reason is that the original developers didn't know any better.

Later as they tried to add features they had to try to make it backward compatible. This is where a lot of the complexity lies. There are lots of crazy workarounds for things that would be simple if you allowed yourself to redesign the file format. It's pretty clear that this was mandated by management, because no software developer would put themselves through that hell for no reason.

Later they added a fast-save feature (I forget what it is actually called). This appends changes to the file without changing the original file. The way they implemented this was really ingenious, but complicates the file structure a lot.

One thing I feel I must point out (I remember posting a huge thing on slashdot when this article was originally posted) is that 2 way file conversion is next to impossible for word processors. That's because the file formats do not contain enough information to format the document. The most obvious place to see this is pagination. The file format does not say where to paginate a text flow (unless it is explicitly entered by the user). It relies of the formatter to do it. Each word processor formats text completely differently. Word, for example famously paginates footnotes incorrectly. They can't change it, though, because it will break backwards compatibility. This is one of the only reasons that Word Perfect survives today – it is the only word processor that paginates legal documents the way the US Department of Justice requires.

Just considering the pagination issue, you can see what the problem is. When reading a Word document, you have to paginate it like Word – only the file format doesn't tell you what that is. Then if someone modifies the document and you need to resave it, you need to somehow mark that it should be paginated like Word (even though it might now have features that are not in Word). If it was only pagination, you might be able to do it, but practically everything is like that.

I recommend reading (a bit of) the XML Word file format for those who are interested. You will see large numbers of flags for things like "Format like Word 95". The format doesn't say what that is – because it's pretty obvious that the authors of the file format don't know. It's lost in a hopeless mess of legacy code and nobody can figure out what it does now.

Fun with NULL

Here's another example of this fine feature:

```
#include #include #include #define LENGTH 128

int main(int argc, char **argv) { char *string = NULL;
int length = 0; if (argc > 1) { string = argv[1]; length =
strlen(string); if (length >= LENGTH) exit(1); }

char buffer[LENGTH]; memcpy(buffer, string, length);
buffer[length] = 0;

if (string == NULL) { printf("String is null, so cancel the
launch.\n"); } else { printf("String is not null, so launch the
```

We saw some really bad Intel CPU bugs in 2015 and we should expect to see more in the future

Dan Luu

2015 was a pretty good year for Intel. Their quarterly earnings reports exceeded expectations every quarter. They continue to be the only game in town for the serious server market, which continues to grow exponentially; from the earnings reports of the two largest cloud vendors, we can see that AWS and Azure grew by 80% and 100%, respectively. That growth has effectively offset the damage Intel has seen from the continued decline of the desktop market. For a while, it looked like cloud vendors might be able to avoid the Intel tax by moving their computation onto FPGAs , but Intel bought one of the two serious FPGA vendors and, combined with their fab advantage, they look well positioned to dominate the high-end FPGA market the same way they've been dominating the high-end server CPU market. Also, their fine for anti-competitive practices turned out to be $1.45B, much less than the benefit they gained from their anti-competitive practices 1 .

Things haven't looked so great on the engineering/bugs side of things, though. We've seen a number of fairly serious CPU bugs and it looks like we should expect more in the future. I don't keep track of Intel bugs unless they're so serious that people I know are scrambling to get a patch in because of the potential impact, and I still heard about two severe bugs this year in the last quarter of the year alone. First, there was the bug found by Ben Serebrin and Jan Beulic , which allowed a guest VM to fault in a way that would cause the CPU to hang in a microcode infinite loop, allowing any VM to DoS its host.

Major cloud vendors were quite lucky that this bug was found by a Google engineer, and that Google decided to share its knowledge of the bug with its competitors before publicly disclosing. Black hats spend a lot of time trying to take down major services. I'm actually really impressed by both the persistence and the cleverness of the people who spend their time attacking the companies I work for. If, buried deep in our infrastructure, we have a bit of code running at DPC that's vulnerable to slowdown because of some kind of hash collision, someone will find and exploit that, even if it takes a long and obscure sequence of events to make it happen. If this CPU microcode hang had been found by one of these black hats, there would have been major carnage for most cloud hosted services at the most inconvenient possible time 2 .

Shortly after the Serebrin/Beulic bug was found, a group of people found that running prime95 , a commonly used tool for benchmarking and burn-in, causes their entire system to lock up . Intel's response to this was :

Intel has identified an issue that potentially affects the 6th Gen Intel® Core™ family of products. This issue only occurs under certain complex workload conditions, like those that may be encountered when running applications like Prime95. In those cases, the processor may hang or cause unpredictable system behavior.

which reveals almost nothing about what's actually going on. If you look at their errata list, you'll find that this is typical, except that they normally won't even name the application that was used to trigger the bug. For example, one of the current errata lists has entries like

Certain Combinations of AVX Instructions May Cause Unpredictable System Behavior

AVX Gather Instruction That Should Result in #DF May Cause Unexpected System Behavior

Processor May Experience a Spurious LLC-Related Machine Check During Periods of High Activity

Page Fault May Report Incorrect Fault Information

As we've seen, "unexpected system behavior" can mean that we're completely screwed. Machine checks aren't great either – they cause Windows to blue screen and Linux to kernel panic. An incorrect address on a page fault is potentially even worse than a mere crash, and if you dig through the list you can find a lot of other scary sounding bugs.

And keep in mind that the Intel errata list has the following disclaimer:

Errata remain in the specification update throughout the product's lifecycle, or until a particular stepping is no longer commercially available. Under these circumstances, errata removed from the specification update are archived and available upon request.

Once they stop manufacturing a stepping (the hardware equivalent of a point release), they reserve the right to remove the errata and you won't be able to find out what errata your older stepping has unless you're important enough to Intel.

Anyway, back to 2015. We've seen at least two serious bugs in Intel CPUs in the last quarter 3 , and it's almost certain there are more bugs lurking. Back when I worked at a company that produced Intel compatible CPUs, we did a fair amount of testing and characterization of Intel CPUs; as someone fresh out of school who'd previously assumed that CPUs basically worked, I was surprised by how many bugs we were able to find. Even though I never worked on the characterization and competitive analysis side of things, I still personally found multiple Intel CPU bugs just in the normal course of doing my job, poking around to verify things that seemed non-obvious to me. Turns out things that seem non-obvious to me are sometimes also non-obvious to Intel engineers. As more services move to the cloud and the impact of system hang and reset vulnerabilities increases, we'll see more black hats investing time in finding CPU bugs. We should expect to see a lot more of these when people realize that it's much easier than it seems to find these

Verilog Won & VHDL Lost? — You Be The Judge!

Dan Luu

This is an archived USENET post from John Cooley on a competitive comparison between VHDL and Verilog that was done in 1997.

I knew I hit a nerve. Usually when I publish a candid review of a particular conference or EDA product I typically see around 85 replies in my e-mail "in" box. Buried in my review of the recent Synopsys Users Group meeting, I very tersely reported that 8 out of the 9 Verilog designers managed to complete the conference's design contest yet none of the 5 VHDL designers could. I apologized for the terseness and promised to do a detailed report on the design contest at a later date. Since publishing this, my e-mail "in" box has become a veritable Verilog/VHDL Beirut filling up with 169 replies! Once word leaked that the detailed contest write-up was going to be published in the DAC issue of "Integrated System Design" (formerly "ASIC & EDA" magazine), I started getting phone calls from the chairman of VHDL International, Mahendra Jain, and from the president of Open Verilog International, Bill Fuchs. A small army of hired gun spin doctors (otherwise know as PR agents) followed with more phone calls. I went ballistic when VHDL columnist Larry Saunders had approached the Editor-in-Chief of ISD for an advanced copy of my design contest report. He felt I was "going to do a hatchet job on VHDL" and wanted to write a rebuttal that would follow my article... and all this was happening before I had even written one damned word of the article!

Because I'm an independent consultant who makes his living training and working both HDL's, I'd rather not go through a VHDL Salem witch trial where I'm publically accused of being secretly in league with the Devil to promote Verilog, thank you. Instead I'm going present everything that happened at the Design Contest, warts and all, and let you judge! At the end of court evidence, I'll ask you, the jury, to write an e-mail reply which I can publish in my column in the follow-up "Integrated System Design".

Contestants were given 90 minutes using either Verilog or VHDL to create a gate netlist for the fastest fully synchronous loadable 9-bit increment-by-3 decrement-by-5 up/down counter that generated even parity, carry and borrow.

Of the 9 Verilog designers in the contest, only 1 didn't get to a final gate level netlist because he tried to code a look-ahead parity generator. Of the 8 remaining, 3 had netlists that missed on functional test vectors. The 5 Verilog designers who got fully functional gate-level designs were:

Larry Fiedler NVidea 3.90 nsec 1147 gates Steve Golson Trilobyte Systems 4.30 nsec 1909 gates Howard Landman HaL Computer 5.49 nsec 1495 gates Mark Papamarcos EDA Associates 5.97 nsec 1180 gates Ed Paluch Paluch & Assoc. 7.85 nsec 1514 gates

The surprize was that, during the same time, none of 5 VHDL designers in the contest managed to produce any gate level designs.

Not VHDL Newbies vs. Verilog Pros

The first reaction I get from the VHDL bigots (who weren't at the competition) is: "Well, this is obviously a case where Verilog veterans whipped some VHDL newbies. Big deal." Well, they're partially right. Many of those Verilog designers are damned good at what they do — but so are the VHDL designers!

I've known Prasad Paranjpe of LSI Logic for years. He has taught and still teaches VHDL with synthesis classes at U. C. Santa Cruz University Extention in the heart of Silicon Valley. He was VP of the Silicon Valley VHDL Local Users Group. He's been a full time ASIC designer since 1987 and has designed real ASIC's since 1990 using VHDL & Synopsys since rev 1.3c. Prasad's home e-mail address is "vhdl@ix.netcom.com" and his home phone is (XXX) XXX-VHDL. ASIC designer Jan Decaluwe has a history of contributing insightful VHDL and synthesis posts to ESNUG while at Alcatel and later as a founder of Easics, a European ASIC design house. (Their company motto: "Easics - The VHDL Design Company".) Another LSI Logic/VHDL contestant, Vikram Shrivastava, has used the VHDL/Synopsys design approach since 1992. These guys aren't newbies!

I followed a double blind approach to putting together this design contest. That is, not only did I have Larry Saunders (a well known VHDL columnist) and Yatin Trivedi (a well known Verilog columnist), both of Seva Technologies comment on the design contest — unknown to them I had Ken Nelsen (a VHDL oriented Methodology Manager from Synopsys) and Jeff Flieder (a Verilog based designer from Ford Microelectronics) also help check the design contest for any conceptual or implementation flaws.

My initial concern in creating the contest was to not have a situation where the Synopsys Design Compiler could quickly complete the design by just placing down a DesignWare part. Yet, I didn't want to have contestants trying (and failing) to design some fruity, off-the-wall thingy that no one truely understood. Hence, I was restricted to "standard" designs that all engineers knew — but with odd parameters thrown in to keep DesignWare out of the picture. Instead of a simple up/down counter, I asked for an up-by-3 and down-by-5 counter. Instead of 8 bits, everything was 9 bits.

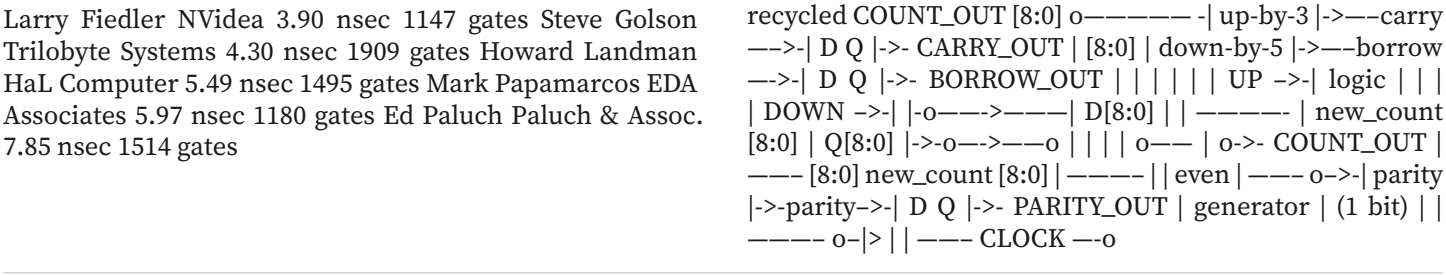

Fig 1) Basic block diagram outlining design's functionality

Cocktail party ideas

Dan Luu

You don't have to be at a party to see this phenomenon in action, but there's a curious thing I regularly see at parties in social circles where people value intelligence and cleverness without similarly valuing on-the-ground knowledge or intellectual rigor. People often discuss the standard trendy topics (some recent ones I've observed at multiple parties are how to build a competitor to Google search and how to solve the problem of high transit construction costs) and explain why people working in the field today are doing it wrong and then explain how they would do it instead. I occasionally have good conversations that fit that pattern (with people with very deep expertise in the field who've been working on changing the field for years), but the more common pattern is that someone with cocktail-party level knowledge of a field will give their ideas on how the field can be fixed.

Asking people why they think their solutions would solve valuable problems in the field has become a hobby of mine when I'm at parties where this kind of superficial pseudo-technical discussion dominates the party. What I've found when I've asked for details is that, in areas where I have some knowledge, people generally don't know what sub-problems need to be solved to solve the problem they're trying to address, making their solution hopeless . After having done this many times, my opinion is that the root cause of this is generally that many people who have a superficial understanding of topic assume that the topic is as complex as their understanding of the topic instead of realizing that only knowing a bit about a topic means that they're missing an understanding of the full complexity of a topic.

Since I often attend parties with programmers, this means I often hear programmers retelling their cocktail-party level understanding of another field (the search engine example above notwithstanding). If you want a sample of similar comments online, you can often see these when programmers discuss "trad" engineering fields. An example I enjoyed was this Twitter thread where Hillel Wayne discussed how programmers without knowledge of trad engineering often have incorrect ideas about what trad engineering is like , where many of the responses are from programmers with little to no knowledge of trad engineering who then reply to Hillel with their misconceptions. When Hillel completed his crossover project , where he interviewed people who've worked in a trad engineering field as well as in software, he got even more such comments . Even when people are warned that naive conceptions of a field are likely to be incorrect, many can't help themselves and they'll immediately reply with their opinions about a field they know basically nothing about.

Anyway, in the crossover project, Hillel compared the perceptions of people who'd actually worked in multiple fields to pop-programmer perceptions of trad engineering. One of the many examples of this that Hillel gives is when people talk about bridge building, where he notes that programmers say things like

The predictability of a true engineer's world is an enviable thing. But ours is a world always in flux, where the laws of physics change weekly. If we did not quickly adapt to the unforeseen, the only foreseeable event would be our own destruction.

and

No one thinks about moving the starting or ending point of the bridge midway through construction.

But Hillel interviewed a civil engineer who said that they had to move a bridge! Of course, civil engineers don't move bridges as frequently as programmers deal with changes in software but, if you talk to actual, working, civil engineers, many civil engineers frequently deal with changing requirements after a job has started that's not fundamentally different from what programmers have to deal with at their jobs. People who've worked in both fields or at least talk to people in the other field tend to think the concerns faced by engineers in both fields are complex, but people with a cocktail-party level of understanding of the field often claim that the field they're not in is simple, unlike their field.

A line I often hear from programmers is that programming is like "having to build a plane while it's flying", implicitly making the case that programming is harder than designing and building a plane since people who design and build planes can do so before the plane is flying 1 . But, of course, someone who designs airplanes could just as easily say "gosh, my job would be very easy if I could build planes with 4 9s of uptime and my plane were allowed to crash and kill all of the passengers for 1 minute every week". Of course, the constraints on different types of projects and different fields make different things hard, but people often seem to have a hard time seeing constraints other fields have that their field doesn't. One might think that understanding that their own field is more complex than an outsider might naively think would help people understand that other fields may also have hidden complexity, but that doesn't generally seem to be the case.

If we look at the rest of the statement Hillel was quoting (which is from the top & accepted answer to a stack exchange question), the author goes on to say:

It's much easier to make accurate projections when you know in advance exactly what you're being asked to project rather than making guesses and dealing with constant changes.

The vast majority of bridges are using extremely tried and true materials, architectures, and techniques. A Roman engineer could be transported two thousand years into the future and generally recognize what was going on at a modern construction site. There would be differences, of course, but you're still building arches for load balancing, you're still using many of the same materials, etc. Most

Windows for Linux Nerds

Jessie Frazelle

I recently started a job at Microsoft. In my first week I have already learned so much about Windows, I figured I would try to put it all into writing. This post is coming to you from a Windows Subsystem for Linux console!

I'm headed to Seattle because I'M JOINING MICROSOFT, at the airport wearing this awesome shirt from @listonb & @Taylorb_msft ���� pic.twitter.com/8rnAg1dsPd

— jessie frazelle (@jessfraz) September 4, 2017

New job and I got a Windows computer and a Linux computer! If you are new to my blog, let me tell you: I love setting up a perfect desktop experience. I've written a few posts on it (for Linux), you should check them out. Setting up a Windows computer is something I have not done in quite some time. I will describe a bit how to set up a windows machine in a reproducible way at the end of this post.

I would like to thank Rich Turner , John Starks , Taylor Brown , and Sarah Cooley for taking the time to explain a lot of the following to me. :)

Windows Subsystem for Linux (WSL)

Let's start with Windows Subsystem for Linux, aka WSL. Even @monkchips wrote that since I joined Microsoft "Linux Subsystem for Windows will definitely be getting a workout." I am super excited about Windows Subsystem for Linux. It is one of the coolest pieces of tech I've seen since I started using Docker.

First, a little background on how WSL works...

You can learn a lot more about this from the Windows Subsystem for Linux Overview . I will go over some of the parts I found to be the most interesting.

The Windows NT kernel was designed from the beginning to support running POSIX, OS/2, and other subsystems. In the early days, these were just user-mode programs that would interact with ntdll to perform system calls. Since the Windows NT kernel supported POSIX there was already a fork system call implemented in the kernel. However, the Windows NT call for fork , NtCreateProcess , is not directly compatible with the Linux syscall so it has some special handling you can read about more under System Calls .

There are both user and kernel mode parts to WSL. Below is a diagram showing the basic Windows kernel and user modes alongside the WSL user and kernel modes.

The blue boxes represent kernel components and the green boxes are Pico Processes. The LX Session Manager Service handles the life cycle of Linux instances. LXCore and lxsys, lxcore.sys and lxss.sys respectively, translate the Linux syscalls into NT APIs.

As you can see in the diagram above, init and /bin/bash are Pico processes. Pico processes work by having system calls and user mode exceptions dispatched to a paired driver. Pico processes and drivers allow Windows Subsystem for Linux to load executable ELF binaries into a Pico process' address space and execute them on top of a Linux-compatible layer of system calls.

You can read even more in depth on this from the MSDN Pico Processes post .

One of the first things I did in WSL was run a syscall fuzzer. I knew it would break but it was interesting for the purposes of figuring out which syscalls had been implemented without looking at the source. This was how I realized PID and mount namespaces were already implemented into clone and unshare !

The WSL kernel drivers, lxss.sys and lxcore.sys , handle the Linux system call requests and translate them to the Windows NT kernel. None of this code came from the Linux kernel, it was all re-implemented by Windows engineers. This is truly mind blowing.

When a syscall is made from a Linux executable it gets passed to lxcore.sys which will translate it into the equivalent Windows NT call. For example, open to NtOpenFile and kill to NTTerminateProcess . If there is no mapping then the Windows kernel mode driver will handle the request directly. This was the case for fork , which has lxcore.sys prepare the process to be copied and then call the appropriate Windows NT kernel APIs to create and copy the process.

You can learn more from the MSDN System Calls post .

Since WSL allows for running Linux binaries natively (without a VM), this allows for some really fun interactions.

You can actually spawn Windows binaries from WSL. Linux ELF binaries get handled by lxcore.sys and lxss.sys as described above and Windows binaries go through the typical Windows userspace.

You can even launch Windows GUI apps as well this way! Imagine a Linux setup where you can launch PowerPoint without a VM.... well this is it!!

Launching X Applications

You can also run X Applications in WSL. You just need an X server. I used vcxsrv to try it out. I run i3 on all my Linux machines and tried it out in WSL like my awesome coworker Brian Ketelsen did in his blog post .

The hidpi is a little gross but if you play with the settings for the X server you can get it to a tolerable place. While I think this is neat for running whatever X applications you love, personally I am going to stick to using tmux as my entrypoint for WSL and using the Windows GUI apps I need vs. Linux X applications. This just feels less heavy (remember, I love minimal) and I haven't come across an X application

You might not need Kubernetes

Jessie Frazelle

I have realized recently that a lot of people think I am just a shill for Kubernetes and I am not. What I have done is write a few blog posts on some interesting problems to be solved in Kubernetes. But I would like to emphasize that those problems are pretty exclusive to the way Kubernetes was designed and you could easily build your own orchestrator without them.

If you need an example of a custom, minimal orchestrator with containerd you should checkout stellar .

Or see my design doc for a multi-tenant orchestrator .

I'll let you dive into that in your own time though. Let's take a new look at a blog post I wrote about Building images securely on Kubernetes .

I feel like I should have more clearly stated how this problem is pretty exclusive to Kubernetes. It's also not really a hard problem. The hard problem I was solving in that post was not how to build images on Kubernetes but how to build images as an unprivileged user in Linux. That is a hard problem. And a serious problem for companies who don't allow root on their machines.

The easier choice if all you need to do is build an image and you are already using containerd, is to run buildkit on the same machine and then you can use the buildkit API library to build your dockerfiles.

Or just run docker-in-docker, I have done this for years on my CI with absolutely no problems.

Anyways, the point I am trying to make is you should use whatever is the easiest thing for your use case and not just what is popular on the internet. With complexity comes a steep learning curve and with a massive number of pluggable layers comes yak

The Brutally Honest Guide to Docker Graphdrivers

Jessie Frazelle

Sup, let me give you fair warning here. Everything contained in this post is my opinion so don't go getting your panties all in a knot on Hacker News because you don't agree with me. I could honestly care less, because that's the thing about my opinion , it's mine.

I am going to give you my honest and dare I say it "blunt" opinion about each of the Docker graphdrivers so you can decide for yourself which one is the best one for you. None are perfect each has it's flaws and I will be laying those out. Let's begin.

Overlayfs was added in the 3.18 kernel. This is important to note because if you are running overlay on an older kernel than 3.18 you are either:

Not running the same overlay.

Running a kernel with overlayfs backported onto it, which is what we call a "frankenkernel".

Frankenkernels are not to be trusted. This is not to say it won't work, hey it might work great, but it's not to be trusted.

Overlay is great but you need a recent kernel. There are also some super obscure kernel bugs with regard to sockets or certain python packages docker/docker#12080 . But I will say personally I use overlay, I have not hit these bugs recently and I have all my 100+ dockerfiles running as continuous builds on my server with overlay and they all work.

Aufs is another great one. But it is not in the kernel by default which blows. On Ubuntu/Debian distros this is as easy as installing the kernel extras package but other distros it might not be as simple.

Btrfs is great too but you need to partition the disk you will use for /var/lib/docker as btrfs first. This is kinda a hurdle to jump that I don't think a lot of people are will-

IN BRIEF

Mike Ash, Mike Ash (Friday Q&A): I'm afraid I once again don't have a Friday Q&A for you today, but I wrote up the best new features in Swift 4 for the Plausible Labs blog, which is almost as good. Check it out over there! (Read More)

Mike Ash, Mike Ash (Friday Q&A): In my last article I discussed the basics of Swift's new Codable protocol, briefly discussed how to implement your own encoder and decoder, and promised another article about a custom binary coder I've been working on. Today, I'm going to present that binary coder. (Read More)

The Signal

Vol. 1, No. 051 · 2026-03-30

Curated and composed automatically.